



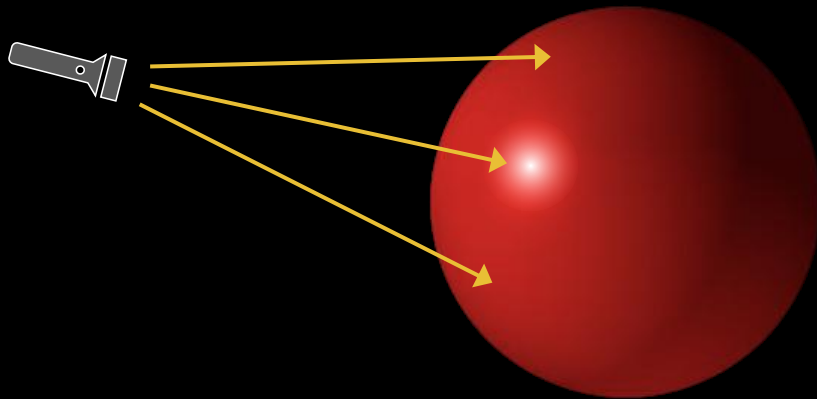
Computação Gráfica

LUZES E MATERIAIS

Judson Santiago

INTRODUÇÃO

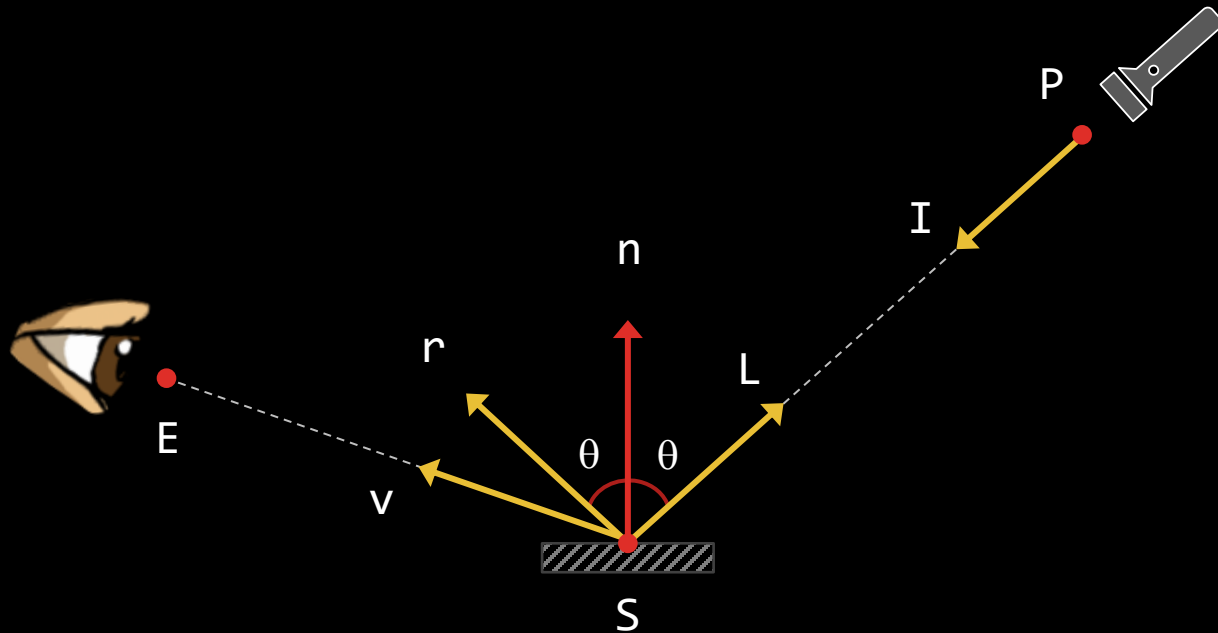
- A **iluminação** de uma cena é obtida:
 - Pela definição das **características dos materiais**
 - Pelo **posicionamento de luzes**



As propriedades do material devem determinar **como a luz interage** com a superfície do objeto

INTRODUÇÃO

- **Luzes** são representadas por vetores
 - O modelo de iluminação local lida com muitos vetores



Legenda

P	Posição da luz
E	Posição do olho
S	Ponto na superfície
I	Direção incidente
L	Vetor luz
n	Vetor normal
r	Vetor reflexão
v	Linha de visão

INTRODUÇÃO

- A cor de cada pixel agora é calculada
 - A equação é aplicada em cada objeto
 - Para cada ponto da superfície

$$c = \underbrace{A_L \otimes m_d}_{\text{Ambiente}} + \underbrace{\max(L \cdot n, 0) \cdot B_L \otimes m_d}_{\text{Difusa}} + \underbrace{\max(L \cdot n, 0) \cdot B_L \otimes R_F(\alpha_h) \frac{m+8}{8} (n \cdot h)^m}_{\text{Especular}}$$

INTRODUÇÃO

- O **Input Layout** não precisa mais da cor dos vértices
 - A cor é calculada no Pixel Shader

```
D3D12_INPUT_ELEMENT_DESC inputLayout[2] =  
{  
    { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0, D3D12_INPUT_PER_VERTEX_DATA, 0 },  
    { "NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12, D3D12_INPUT_PER_VERTEX_DATA, 0 }  
};
```



MATERIAIS

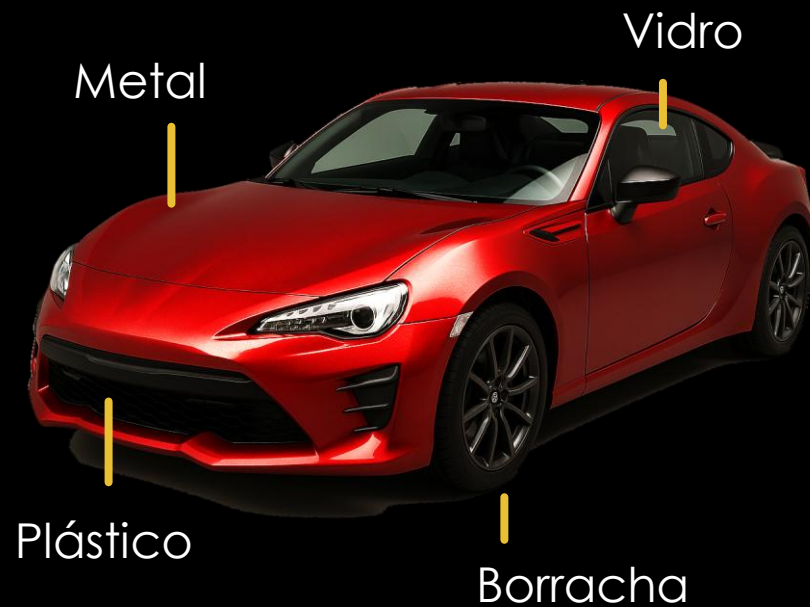
- A **modelagem** de **materiais realistas** requer achar valores para:
 - A cor refletida (albedo difuso)
 - A refletividade (reflexão de Fresnel)
 - Rugosidade

Alguns **ajustes artísticos** podem ser necessários para obter os melhores resultados

```
struct Material
{
    XMFLOAT4 Albedo;           // cor da luz refletida
    XMFLOAT3 Fresnel;          // valor R0 do Efeito Fresnel
    float Roughness;           // fator de rugosidade da superfície
};
```

MATERIAIS

- O **material** pode **variar** ao longo da superfície



Como atribuir
materiais para um
modelo 3D?

A solução prevalente é
definir um material por
objeto ou para cada
textura

MATERIAIS

- Ainda **não** temos **texturas**
- Vamos usar um material por **objeto**

```
Material silver;  
silver.Albedo = XMFLOAT4(0.50f, 0.50f, 0.50f, 1.0f);  
silver.Fresnel = XMFLOAT3(0.95f, 0.93f, 0.88f);  
silver.Roughness = 0.05f;
```

```
Material gold;  
gold.Albedo = XMFLOAT4(1.00f, 0.71f, 0.29f, 1.0f);  
gold.Fresnel = XMFLOAT3(1.0f, 0.71f, 0.29f);  
gold.Roughness = 0.60f;
```

```
Material plastic;  
plastic.Albedo = XMFLOAT4(0.05f, 0.05f, 0.05f, 1.0f);  
plastic.Fresnel = XMFLOAT3(0.05f, 0.05f, 0.05f);  
plastic.Roughness = 0.95f;
```

Os materiais serão
utilizados no Pixel
Shader.

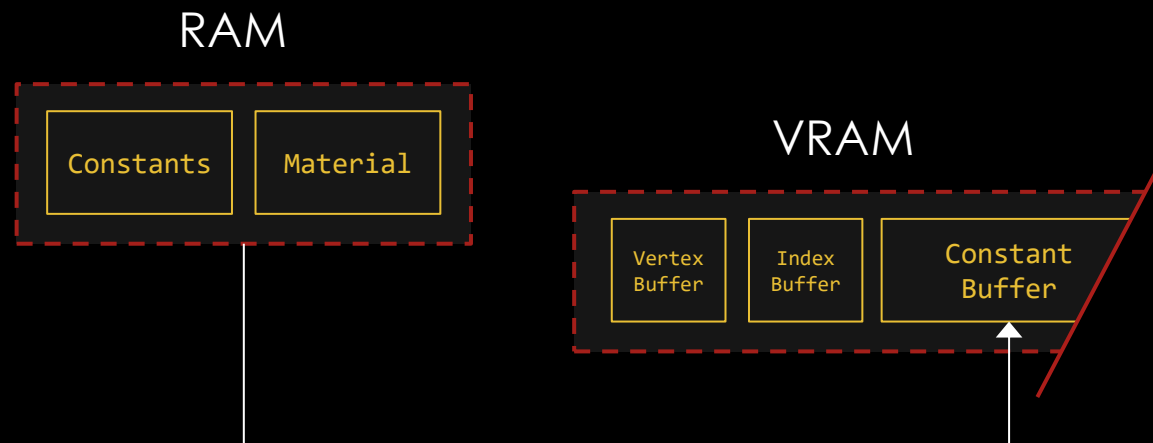
Eles podem
ser alterados a
cada desenho.

MATERIAIS

- Os **materiais** precisam ser carregadas em VRAM
 - Precisam ser acessados pelos **Shaders** na GPU
 - Devem ser carregados em Constant Buffers

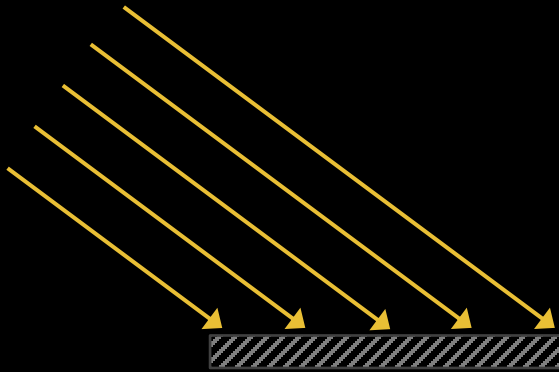
```
struct Constants
{
    XMFLOAT4x4 World;
    ...
};

struct Material
{
    XMFLOAT4 Albedo;
    XMFLOAT3 Fresnel;
    float Roughness;
};
```

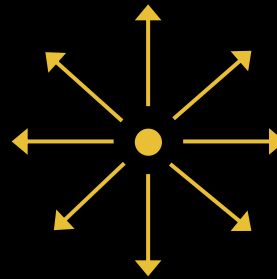


FONTES DE LUZ

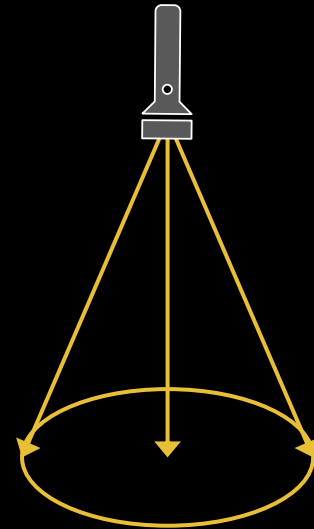
- Já vimos como **modelar a luz**
 - Mas existem diferentes **tipos de fontes de luz**



Luz Paralela



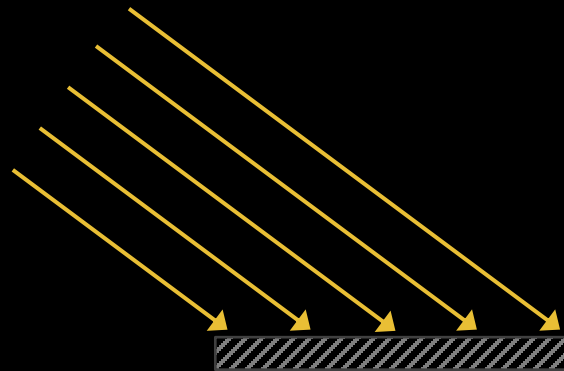
Luz Ponto



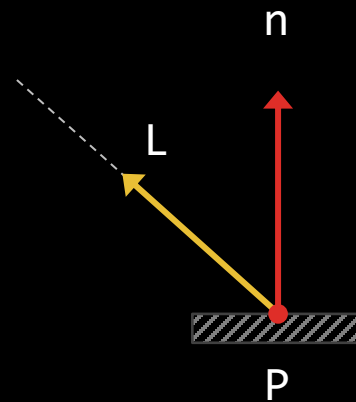
Luz Holofote

LUZ PARALELA

- Simula uma fonte de luz que está muito distante
 - Os raios de luz podem ser considerados paralelos
 - Os efeitos da distância podem ser ignorados



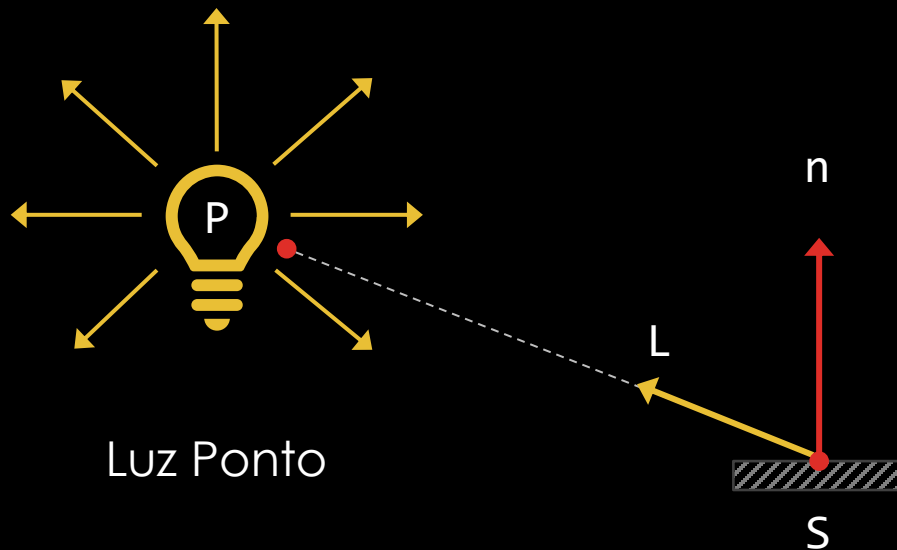
Luz Paralela



Uma fonte de luz paralela é definida por um vetor

LUZ PONTO

- Uma **fonte** que **irradia luz em todas as direções**
 - Como acontece com uma **lâmpada**



Para todo ponto S na superfície, existe um raio de luz originado em P

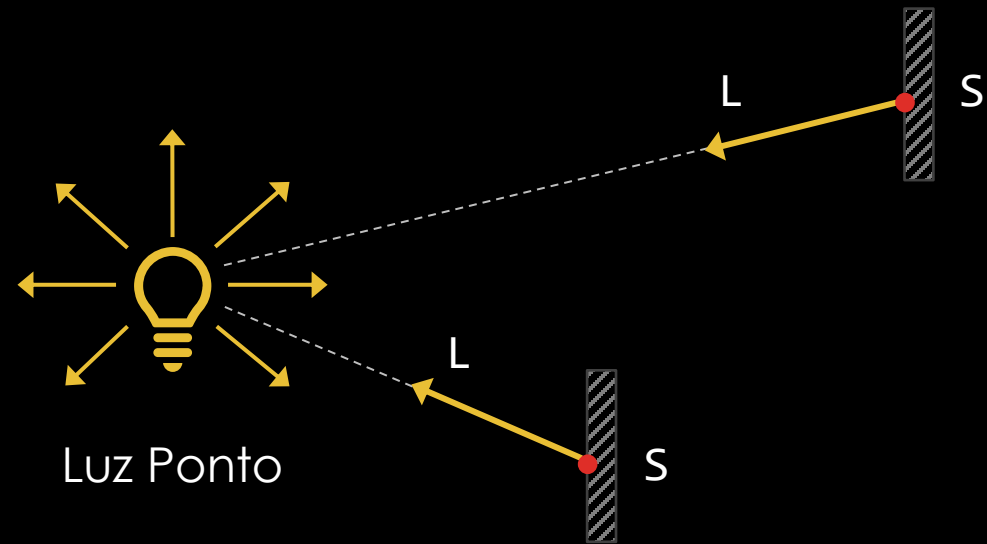
$$L = \frac{P - S}{\|P - S\|}$$

LUZ PONTO

- A **intensidade da luz** se reduz com a distância
 - Fisicamente é modelado pelo **inverso do quadrado**

$$I(d) = \frac{I_0}{d^2}$$

- A fórmula funciona bem para:
 - Valores baseados na física
 - High Dynamic Range (HDR)
 - Tone Mapping



LUZ PONTO

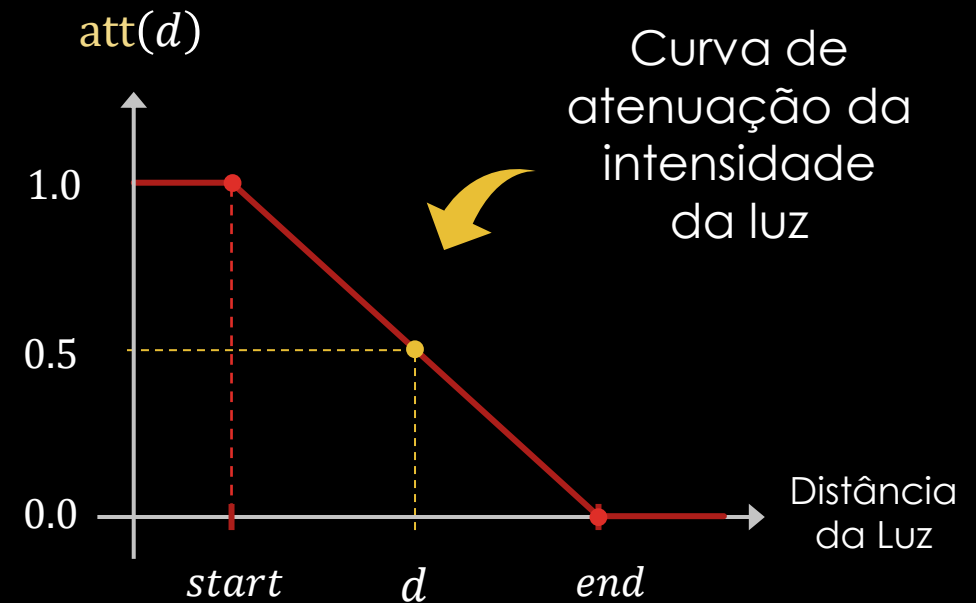
- Porém, uma **formulação mais simples** é comumente utilizada:

- A intensidade da luz é multiplicada por uma **função de atenuação**

$$\text{att}(d) = \text{saturnate} \left(\frac{\text{end} - d}{\text{end} - \text{start}} \right)$$

- Sendo:

$$\text{saturnate}(x) = \begin{cases} x, & 0 \leq x \leq 1 \\ 0, & x < 0 \\ 1, & x > 1 \end{cases}$$



Ex.: 20 70 120

LUZ PONTO

- A luz originada em um **ponto de luz** sofre atenuação
 - A luz recebida deve ser ajustada pelo fator de atenuação

Luz Recebida

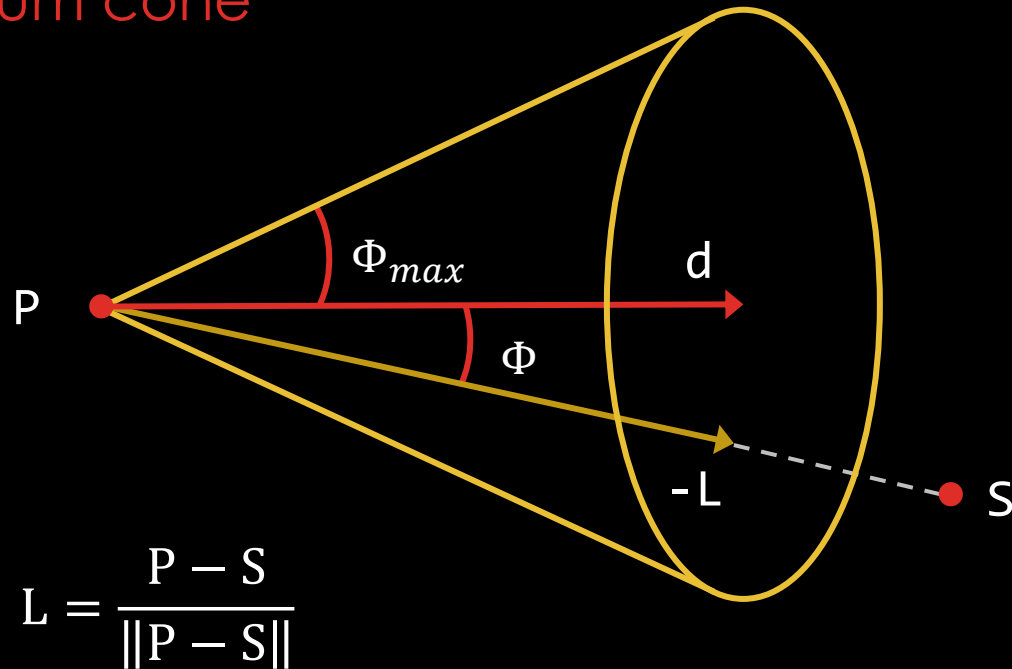
$$c = A_L \otimes m_d + \max(L \cdot n, 0) \cdot att(d) \cdot B_L \otimes \left(m_d + R_F(\alpha_h) \frac{m + 8}{8} (n \cdot h)^m \right)$$

Objetos distantes não recebem luz, o que fornece uma otimização para os cálculos de iluminação

LUZ HOLOFOTE

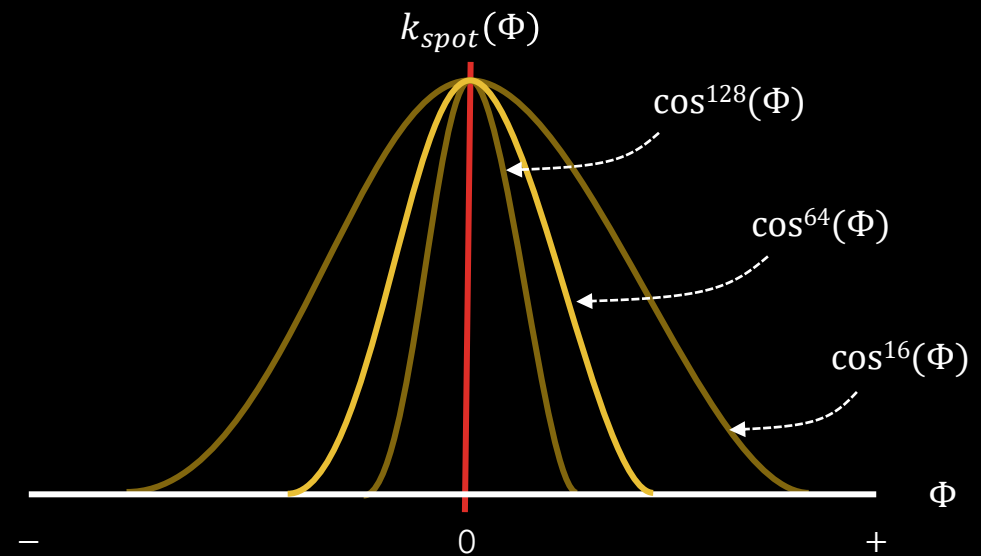
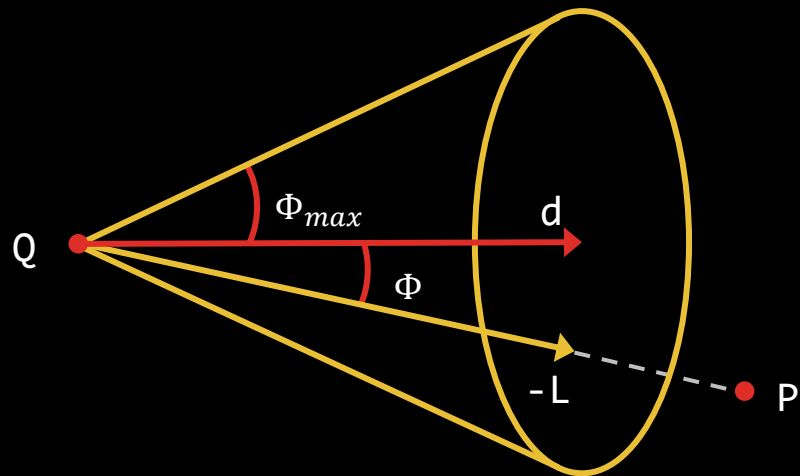
- Um holofote **irradia luz através de um cone**
 - Com abertura Φ_{max}
 - Em uma direção d

O ponto **S** recebe luz apenas se o ângulo Φ entre d e $-L$ é menor que Φ_{max}



LUZ HOLOFOTE

- Podemos usar uma função para controlar:
 - A variação de intensidade de luz
 - O tamanho do cone



$$k_{spot}(\Phi) = \max(\cos \Phi, 0)^S = \max(-L \cdot d, 0)^S$$

LUZ HOLOFOTE

- A **luz recebida** sofre **atenuação** e o **efeito holofote**

Luz Recebida

$$c = A_L \otimes m_d + \max(L \cdot n, 0) \cdot att(d) \cdot k_{spot}(\Phi) \cdot B_L \otimes \left(m_d + R_F(\alpha_h) \frac{m+8}{8} (n \cdot h)^m \right)$$

A **luz de holofote** é **computacionalmente mais cara** que a luz ponto, que por sua vez é mais cara que a luz paralela

LUZES

- Para criar uma cena com várias luzes
 - É necessário reorganizar os registros

```
struct Scene
{
    XMFLOAT4X4 ViewProj;
    XMFLOAT4 Ambient;
    XMFLOAT3 Eye;
    int Amb;
    int Dif;
    int Spe;
    int Lam;
    int Mat;
    Light Lights[3];
};

struct Light
{
    XMFLOAT3 Direction; // vetor na direção da luz
    float FalloffStart; // início da atenuação
    XMFLOAT3 Strength; // potência da luz
    float FalloffEnd; // limite da atenuação
    XMFLOAT3 Position; // posição da luz
    float SpotPower; // potência do holofote
};
```



LUZES

- O registro vai **representar qualquer tipo de luz**



Paralela

Direção
Intensidade



Ponto

Direção
Intensidade
Posição
Atenuação



Holofote

Direção
Intensidade
Posição
Atenuação
Potência

```
struct Light
{
    XMFLOAT3 Direction;
    float FalloffStart;
    XMFLOAT3 Strength;
    float FalloffEnd;
    XMFLOAT3 Position;
    float SpotPower;
};
```

IMPLEMENTAÇÃO

- Mas por que essa **organização estranha do registro**?
 - Ele deve seguir a regra de empacotamento do **HLSL**
 - Elementos devem ser alinhados em 16 bytes (vetores 4D)

```
struct Light
{
    XMFLOAT3 Direction;    // vetor na direção da luz
    float     FalloffStart; // início da atenuação
    XMFLOAT3 Strength;     // potência da luz
    float     FalloffEnd;  // limite da atenuação
    XMFLOAT3 Position;     // posição da luz
    float     SpotPower;    // potência do holofote
};
```

IMPLEMENTAÇÃO

- Em **HLSL**, os registros são alinhados em blocos de 16 bytes
 - Membros não podem ser divididos em dois blocos
 - É preciso fazer **padding** para manter compatibilidade

```
// HLSL
cbuffer Data: register (b0)
{
    float3 Pos;
    float3 Dir;
};

// alinhamento na memória
1: (Pos.x, Pos.y, Pos.z, vazio)
2: (Dir.x, Dir.y, Dir.z, vazio)
```

```
// C++
struct Data
{
    XMFLOAT3 Pos;
    XMFLOAT3 Dir;
};

// alinhamento na memória
1: (Pos.x, Pos.y, Pos.z, Dir.x)
2: (Dir.y, Dir.z, vazio, vazio)
```

IMPLEMENTAÇÃO

- O **padding** garante o acesso correto nos **Shaders**
 - Dados podem ser **copiados** diretamente

```
// HLSL
cbuffer Data: register (b0)
{
    float3 Pos;
    float __pad0;
    float3 Dir;
    float __pad1;
};

// alinhamento na memória
1: (Pos.x, Pos.y, Pos.z, __pad0)
2: (Dir.x, Dir.y, Dir.z, __pad1)
```

```
// C++
struct Data
{
    XMFLOAT3 Pos;
    float __pad0;
    XMFLOAT3 Dir;
    float __pad1;
};

// alinhamento na memória
1: (Pos.x, Pos.y, Pos.z, __pad0)
2: (Dir.x, Dir.y, Dir.z, __pad1)
```


IMPLEMENTAÇÃO

- O registro **Light** não precisa de padding
 - A ordem dos elementos foi cuidadosamente escolhida

```
struct Light
{
    XMFLOAT3 Direction;    // vetor na direção da luz
    float     FalloffStart; // início da atenuação
    XMFLOAT3 Strength;     // potência da luz
    float     FalloffEnd;  // limite da atenuação
    XMFLOAT3 Position;     // posição da luz
    float     SpotPower;   // potência do holofote
};

// alinhamento na memória
1: (Direction.x, Direction.y, Direction.z, FalloffStart)
2: (Strength.x, Strength.y, Strength.z, FalloffEnd)
3: (Position.x, Position.y, Position.z, SpotPower)
```

RESUMO

- A **iluminação** consiste em **calcular a cor das superfícies** dos objetos
 - Com base nos dados de luzes e materiais
 - **Luzes** podem ser de 3 tipos
paralelas, pontos, holofotes
 - **Materiais** possuem propriedades
albedo difuso, fator de Fresnel, rugosidade
- A **implementação** pode ser realizada eficientemente no **Pixel Shader**
 - Vários pixels são **processados em paralelo** na GPU